Running Threestudio in Habrok

Step 0: Setup Habrok for VScode

Install the “Remote - SSH” extension
In vscode do: ctrl + shift +p and type

```
Unset
ssh -X username@interactive1.hb.hpc.rug.nl
```

Step 1: Clone Threestudio in Habrok

Open a terminal and clone threestudio

```
Unset
git clone https://github.com/threestudio-project/threestudio.git
cd threestudio
```

Step 2: Allocate a GPU interactively

Open a terminal and type:

```
Unset
srun --partition=gpu --gres=gpu:1 --time=2:00:00 --pty bash
```

Wait until you get a gpu

Step 3: Load CUDA and other modules

Open a terminal and type:

```
Unset
module purge
module load CUDA/11.8.0
module load Python/3.10.8
```

```
module load GCC/11.3.0

export
CUDA_HOME=/cvmfs/hpc.rug.nl/versions/2023.01/rocky8/x86_64/amd/zen3/software/CUDA/11.8.0
export PATH=$CUDA_HOME/bin:$PATH
export LD_LIBRARY_PATH=$CUDA_HOME/lib64:$LD_LIBRARY_PATH
export CC=gcc export CXX=g++

nvcc --version
python --version
gcc --version
echo $CUDA_HOME
module list

which python
which pip
pip list | grep torch
```

Setup all the modules inside the gpu first

Step 4: Create a virtual environment

Set up a Python virtual environment to keep your dependencies isolated:

```
Unset
python -m venv threestudio
source threestudio/bin/activate
```

Step 5: Install PyTorch and Torchvision

Set up a Python virtual environment to keep your dependencies isolated:

```
Unset

pip install torch==2.5.1+cu118 torchvision==0.20.1+cu118
--index-url https://download.pytorch.org/whl/cu118
```

Step 6: Set Up Python Environment Variables

Add the necessary paths to ensure Python can locate the installed packages:

NOTE: make sure to check what home number you have and change accordingly

```
Unset
export PYTHONUSERBASE=/home2/$(whoami)/.local

export ENABLE_USER_SITE=True

export
PYTHONPATH=/home2/$(whoami)/.local/lib/python3.10/site-packages:$
PYTHONPATH
```

Step 7: Verify the Installation

Verify that PyTorch is installed and can access the GPU:

```
Unset
python -c "import torch; print(torch.__version__);
print(torch.cuda.is_available())"
```

Expected Output:

```
2.5.1+cu118

True
```

If you see `True` for `torch.cuda.is_available()`, you're good to go!

Step 8: Get the requirements needed

NOTE: there are some dependency issues with their current requirements.txt file so please update it (delete everything in there and paste the following) before install anything else:

```
Unset

lightning==2.0.0

omegaconf==2.3.0

jaxtyping==0.2.36

typeguard==4.4.1

git+https://github.com/KAIR-BAIR/nerfacc.git@v0.5.2

git+https://github.com/NVlabs/tiny-cuda-nn/#subdirectory=bindings/torch

diffusers==0.19.3

transformers==4.28.1

accelerate==1.2.1

opencv-python==4.10.0.84

tensorboard==2.18.0

matplotlib==3.10.0

imageio==2.36.1

imageio-ffmpeg==0.5.1

git+https://github.com/NVlabs/nvdiffrast.git

libigl==2.5.1
```

```
xatlas==0.0.9

trimesh==4.5.3

networkx==3.2.1

pysdf==0.1.9

PyMCubes==0.1.6

wandb==0.19.1

gradio==4.11.0

git+https://github.com/ashawkey/envlight.git

torchmetrics==1.6.1

ipython==8.31.0

ipywidgets==8.1.5


# deepfloyd

xformers==0.0.29.post1

bitsandbytes==0.38.1

sentencepiece==0.2.0

safetensors==0.5.0

huggingface-hub==0.21.0


# for zero123

einops==0.8.0
```

```
kornia==0.7.4

taming-transformers-rom1504==0.0.6

git+https://github.com/openai/CLIP.git


#controlnet

controlnet-aux==0.0.9
```

Then install the following in the terminal:

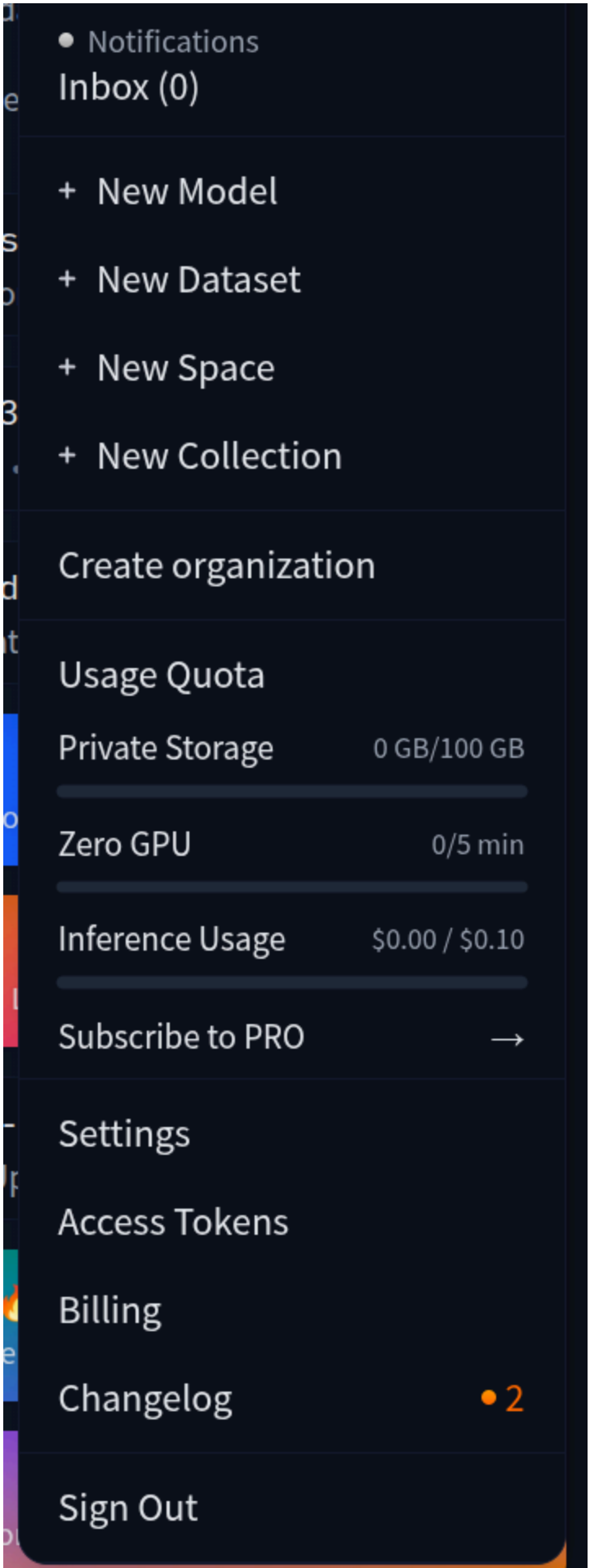
```
Unset

pip install ninja

pip install -r requirements.txt
```

Step 9: Huggingface login

In order to run anything which is ran via prompt you will need a huggingface account.

Make an account using this link: https://huggingface.co/

Then you need to create an access token in here by clicking your profile:


Notifications
Inbox (0)
+ New Model
+ New Dataset
+ New Space
+ New Collection
Create organization
Usage Quota
Private Storage
0 GB/100 GB
Zero GPU
0/5 min
Inference Usage
$0.00 / $0.10
Subscribe to PRO
→
Settings
Access Tokens
Billing
Changelog
2
Sign Out

Click on “Create new token” and make sure to check the “Read access to contents of all public gated repos you can access” in user permissions.

Then you can create your access token, and save it.

Then run this command and paste the token when prompted:

```
Unset
huggingface-cli login
```

This will let you download the weights for models such as dreamfusion.

Step 10: Test the code

Verify that PyTorch is installed and can access the GPU:

```
Unset
# if you have agreed the license of DeepFloyd IF and have >20GB
VRAM
# please try this configuration for higher quality
python launch.py --config configs/dreamfusion-if.yaml --train
--gpu 0 system.prompt_processor.prompt="a zoomed out DSLR photo
of a baby bunny sitting on top of a stack of pancakes"
# otherwise you could try with the Stable Diffusion model, which
fits in 6GB VRAM
python launch.py --config configs/dreamfusion-sd.yaml --train
--gpu 0 system.prompt_processor.prompt="a zoomed out DSLR photo
of a baby bunny sitting on top of a stack of pancakes"
```

Test both to see if it has enough VRAM

Step 11: Reallocate GPU (If Needed)

If your job time is running out, terminate the current session and reallocate a GPU as in Step 1.

To terminate early:

```
exit
```

Step 12: Save the Configuration

Save the environment variables in your `~/.bashrc` or `~/.bash_profile` for future sessions:

NOTE: again make sure to change the home number to your specific one

```
Unset
echo 'export PYTHONUSERBASE=/home2/$(whoami)/.local' >> ~/.bashrc
echo 'export ENABLE_USER_SITE=True' >> ~/.bashrc
echo 'export
PYTHONPATH=/home2/$(whoami)/.local/lib/python3.10/site-packages:$
PYTHONPATH' >> ~/.bashrc
source ~/.bashrc
```

Running threestudio in Habrok after setting it up.

Step 1: Allocate a GPU Node

Start by allocating a GPU node on Habrok:

```
srun --partition=gpu --gres=gpu:1 --time=2:00:00 --pty bash
```

This command allocates a GPU for 4 hours. Adjust the `--time` parameter as needed.

Step 2: Load CUDA and Other Modules

Load the appropriate CUDA version required for your PyTorch version. For PyTorch 2.5.1 with CUDA 11.8:

```
module purge
module load CUDA/11.8.0
module load Python/3.10.8
module load GCC/11.3.0
```

```
export
CUDA_HOME=/cvmfs/hpc.rug.nl/versions/2023.01/rocky8/x86_64/amd/zen3/so
ftware/CUDA/11.8.0
export PATH=$CUDA_HOME/bin:$PATH
export LD_LIBRARY_PATH=$CUDA_HOME/lib64:$LD_LIBRARY_PATH
export CC=gcc export CXX=g++

nvcc --version
python --version
gcc --version
echo $CUDA_HOME
module list

#NOTE again: change this to your venv to activate
source /home2/s4787641/threestudio/threestudio/bin/activate



which python

which pip

pip list | grep torch

python -c "import torch; print(torch.__version__);
print(torch.cuda.is_available())"
```

Automate the process through bashrc

What You Need to Do

You should add the necessary environment variable exports and module loads to automate your setup. Here's how:

Edit `~/.bashrc`: Open the file for editing using your favorite editor:

```
nano ~/.bashrc
```

Add the Automation Settings: Append the following lines to automate the modules and environment variables:

```
# Load required modules
module load Python/3.10.8
module load CUDA/11.8.0
module load GCC/11.3.0


# Activate Threestudio Python environment
source /home2/s4787641/threestudio/threestudio/bin/activate

# CUDA paths
export
CUDA_HOME=/cvmfs/hpc.rug.nl/versions/2023.01/rocky8/x86_64/amd/zen3/so
ftware/CUDA/11.8.0
export PATH=$CUDA_HOME/bin:$PATH
export LD_LIBRARY_PATH=$CUDA_HOME/lib64:$LD_LIBRARY_PATH
```

```
# Python paths
export
PYTHONPATH=/home2/s4787641/threestudio/threestudio/lib/python3.10/site
-packages:$PYTHONPATH
```

Save and Exit: If you're using `nano`, save with `CTRL+O`, press `Enter`, and exit with `CTRL+X`.

Reload `~/.bashrc`: Apply the changes without restarting your session:
`source ~/.bashrc`

Verify Automation

To ensure everything is loaded and set up properly after logging in or starting a new session:

Check module list:
`module list`
Confirm environment variables:
`echo $CUDA_HOME`
`echo $PATH`
`echo $LD_LIBRARY_PATH`
Verify the loaded tools:
`gcc --version`
`python --version`
`nvcc --version`

This setup will ensure everything is ready whenever you start a new session, avoiding manual setup steps every time. Let me know if you encounter any issues!